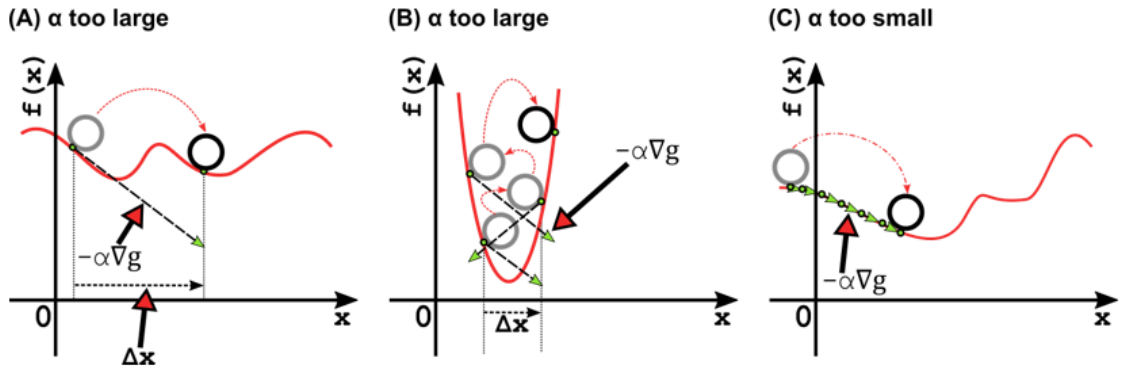



**Figure 16.16 Gradient descent: Δf is the change in the cost function caused by a variation Δx in its parameter.**

By using a learning rate, we can tune[13] this `alpha` hyper-parameter and adapt the optimization algorithm to the function we need to optimize.

An even better solution, however, is to use a variable `alpha`, for example a value that decreases as the steps progresses: initially it's large enough to let the optimization quickly explore a wide area and possibly get out of local minima, and then it gets smaller and smaller, so that, in the end, fine-tuning can be done and oscillation around stationary points (minima) is avoided.

Another great option is to introduce the concept of momentum: instead of basing the next step just on last gradient, gradient descent with momentum smooths the update by computing the delta as a linear combination of the last few gradients (with older gradients having lower weight on the final result than newer ones).

As the term suggests, having a momentum (like it happens in kinematics) means that if the algorithm speed was high, meaning that it was updating a coordinate with large steps, then when the slope of the curve changes, the speed will smooth out, but not abruptly.